



**FACULTAD DE INGENIERIA
DEPARTAMENTO DE INGENIERIA DE
SISTEMAS**

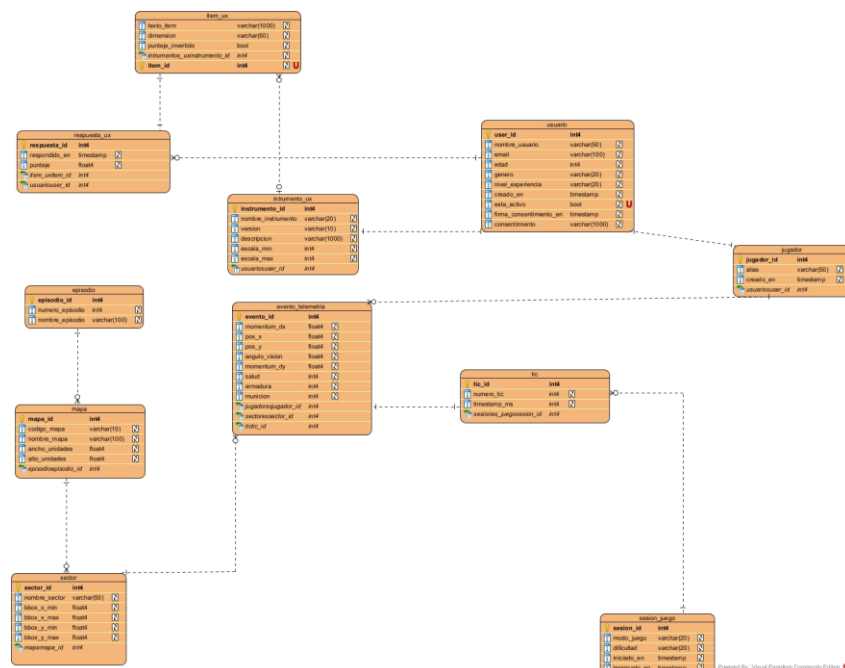
“Bases de Datos”

Ing. Andrés Oswaldo Calderon

***Project: Designing a Telemetry and UX Database for
Chocolate Doom Research***

04/Junio/2026

**Gabriela Melo Gualteros
Nicolás Roa Hernandez
David Galindo Rojas**



El diagrama entidad-relación modela un sistema para capturar y analizar el comportamiento de jugadores en un videojuego, integrando telemetría en tiempo real con evaluaciones de experiencia de usuario (UX). Contiene once entidades organizadas en tres dominios.

El dominio de juego estructura el escenario: **episodio** agrupa mapas, **mapa** define el escenario con sus dimensiones, **sectore** subdivide el mapa en regiones geográficas, **sesion_juego** registra cada partida y **tic** representa la unidad mínima de tiempo dentro de una sesión.

El dominio de telemetría captura el estado del jugador en cada instante.

evento_telemetria almacena posición, ángulo de visión, momentum, salud, armadura y munición, vinculándose a un **tic**, un **sectore** y un **jugadore**.

El dominio UX evalúa la experiencia subjetiva del participante. **usuario** se relaciona con **instrumento_ux**, que contiene múltiples **item_ux**, y las respuestas quedan registradas en **respuesta_ux**.

2. Relational schema:

usuarios(user_id, nombre_usuario, email, edad, genero, nivel_experiencia, creado_en, esta_activo, firma_consentimiento_en, consentimiento)

jugadores(jugador_id, alias, creado_en, usuariosuser_id)

instrumentos_ux(instrumento_id, nombre_instrumento, version, descripcion, escala_min, escala_max)

items_ux(item_id, numero_item, texto_item, dimension, puntaje_invertido, instrumentos_uxinstrumento_id)

respuestas_ux(respuesta_id, respondido_en, puntaje_total, usuariosuser_id, sesiones_juegosesion_id, instrumentos_uxinstrumento_id, items_uxitem_id)

episodios(episodio_id, numero_episodio, nombre_episodio)

mapas(mapa_id, codigo_mapa, nombre_mapa, ancho_unidades, alto_unidades, episodioepisodio_id)

sectores(sector_id, nombre_sector, bbox_x_min, bbox_x_max, bbox_y_min, bbox_y_max, mapasmapa_id)

sesiones_juego(sesion_id, modo_juego, dificultad, iniciado_en, terminado_en, mapasmapa_id)

tic(tic_id, numero_tic, timestamp_ms, sesiones_juegosesion_id)

eventos_telemetria(evento_id, momentum_dx, pos_x, pos_y, angulo_vision, momentum_dy, salud, armadura, municion, jugadoresjugador_id, sectoressector_id, tictic_id)

registros_auditoria(auditoria_id, accedido_por, nombre_tabla, accion, accedido_en, detalles)

El esquema relacional define la estructura lógica de la base de datos mediante doce relaciones. Las claves primarias identifican unívocamente cada tupla, y las claves foráneas garantizan la integridad referencial entre entidades, siguiendo las convención de nombrar cada FK con el prefijo de la tabla que referencia.

El esquema se organiza en tres dominios. El dominio de usuarios e instrumentos UX agrupa las relaciones usuarios, jugadores, instrumentos_ux, items_ux y respuestas_ux, permitiendo registrar participantes, sus perfiles de juego y sus evaluaciones subjetivas de experiencia. El dominio de juego comprende episodios, mapas, sectores, sesiones_juego y tic, modelando la estructura del videojuego y la granularidad temporal de cada partida. La relación eventos_telemetria actúa como eje central del sistema, conectando ambos dominios al referenciar jugadores, sectores y tics de tiempo. Por último, registros_auditoria opera de forma independiente, trazando todas las acciones realizadas sobre la base de datos sin vincularse a ninguna otra relación.

3. ETL description:

timestamp	tic	x	y	z	angle	momx	momy	shield	health	ammo	sector_id
2026-06-03 22:25:07	208	-1	50	0	90.00	-1	2	0	100	50	(0,0)
2026-06-03 22:25:07	215	-1	68	0	90.00	-1	1	0	100	50	(0,0)
2026-06-03 22:25:07	222	-1	79	0	90.00	-1	2	0	100	50	(0,0)
2026-06-03 22:25:07	229	-1	107	0	90.00	-1	5	0	100	50	(0,0)
2026-06-03 22:25:08	236	-1	149	0	90.00	-1	5	0	100	50	(0,0)
2026-06-03 22:25:08	243	-1	177	0	90.00	-1	2	0	100	50	(0,0)
2026-06-03 22:25:08	250	-1	191	0	95.27	-1	1	0	100	50	(0,0)
2026-06-03 22:25:08	257	-2	206	0	102.30	-1	3	0	100	50	(0,0)
2026-06-03 22:25:08	264	-9	240	0	102.30	-2	5	0	100	50	(0,0)
2026-06-03 22:25:09	271	-18	285	0	102.30	-2	7	0	100	50	(0,1)
2026-06-03 22:25:09	278	-29	336	0	104.06	-2	7	0	100	50	(0,1)
2026-06-03 22:25:09	285	-42	390	0	105.82	-2	7	0	100	50	(0,1)
2026-06-03 22:25:09	292	-56	443	8	105.82	-2	5	0	100	50	(0,1)
2026-06-03 22:25:09	299	-64	474	8	105.82	-1	2	0	100	50	(0,1)
2026-06-03 22:25:10	306	-68	490	8	105.82	-1	1	0	100	50	(0,1)
2026-06-03 22:25:10	313	-70	498	8	100.55	-1	0	0	100	50	(0,1)
2026-06-03 22:25:10	320	-71	502	8	79.45	-1	0	0	100	50	(0,2)
2026-06-03 22:25:10	327	-73	502	8	54.84	-2	-2	0	100	50	(0,2)
2026-06-03 22:25:10	334	-89	483	8	30.23	-4	-4	0	100	50	(0,1)
2026-06-03 22:25:11	341	-112	463	8	5.62	-3	-3	0	100	50	(0,1)
2026-06-03 22:25:11	348	-112	451	0	2.11	0	-2	0	100	50	(0,1)
2026-06-03 22:25:11	355	-112	445	0	358.59	0	-1	0	100	50	(0,1)

Muestra de Datos Crudos de Telemetría

La imagen muestra una muestra de los datos crudos capturados durante una sesión de juego el 2026-06-03. Cada fila representa un evento de telemetría registrado cada pocos milisegundos, identificado por un número de tic incremental (208, 215, 222...) con intervalos de 7 unidades. Las columnas registran la posición del jugador en x, y, z, el ángulo de visión, el momentum en x y y, el escudo, la salud, la munición y el sector en que se encontraba expresado en coordenadas (x,y).

ETL

Los datos fueron extraídos directamente del motor del juego en formato tabular. Durante la transformación, el campo de sector fue convertido desde notación de coordenadas hacia el identificador de la tabla `sectores`, y los tipos de dato fueron ajustados según el esquema (FLOAT para posición y momentum, INTEGER para salud, escudo y munición). Finalmente los registros fueron cargados en la tabla `eventos_telemetria` respetando las claves foráneas definidas en el modelo.

4. Queries + results:

-- Query 1: Duracion promedio de sesiones por mapa

SELECT

m.codigo_mapa,

m.nombre_mapa,

ROUND(AVG(EXTRACT(MINUTE FROM (sj.terminado_en -
sj.iniciado_en))), 2) AS duracion_promedio_minutos

FROM sesiones_juego sj

JOIN mapas m ON m.mapa_id = sj.mapasma_id

GROUP BY m.codigo_mapa, m.nombre_mapa

ORDER BY duracion_promedio_minutos DESC;

	codigo_mapa character varying (10)	nombre_mapa character varying (100)	duracion_promedio_minutos numeric
1	E4M1	CATAFALQUE	30.00
2	E1M5	THE CITADEL	30.00
3	E3M1	THE STOREHOUSE	30.00
4	E1M1	THE DOCKS	30.00

-- Query 3: Distancia total de trayectoria por jugador (min y max)

```

SELECT
    j.alias,
    ROUND(CAST(MIN(trayectoria.distancia_total) AS NUMERIC), 2) AS
trayectoria_mas_corta,
    ROUND(CAST(MAX(trayectoria.distancia_total) AS NUMERIC), 2) AS
trayectoria_mas_larga
FROM (
    SELECT
        e.jugadoresjugador_id,
        t.sesiones_juegosesion_id,
        SUM(
            SQRT(
                POWER(e.pos_x - prev.pos_x, 2) +
                POWER(e.pos_y - prev.pos_y, 2)
            )
        ) AS distancia_total
    FROM eventos_telemetria e
    JOIN tic t ON t.tic_id = e.tictic_id
    JOIN eventos_telemetria prev ON prev.jugadoresjugador_id = e.jugadoresjugador_id
        AND prev.tictic_id = e.tictic_id - 1
    GROUP BY e.jugadoresjugador_id, t.sesiones_juegosesion_id
) AS trayectoria
JOIN jugadores j ON j.jugador_id = trayectoria.jugadoresjugador_id
GROUP BY j.alias
ORDER BY trayectoria_mas_larga DESC;

```

	alias character varying (50)	trayectoria_mas_corta numeric	trayectoria_mas_larga numeric
1	Nicolas	167368.80	1524571.51
2	gaby.12	331390.15	331390.15
3	Lindsay	119153.68	119153.68
4	Juanes	108266.51	108266.51
5	Juan	76697.19	76697.19
6	Davidg	67549.87	67549.87

-- Query 4: Respuestas UX de jugadores con mas tics que el promedio

```

SELECT
    j.alias,
    i.texto_item,
    r.puntaje_total
FROM respuestas_ux r
JOIN usuarios u ON u.user_id = r.usuario_id
JOIN jugadores j ON j.usuario_id = u.user_id
JOIN items_ux i ON i.item_id = r.item_id
WHERE j.jugador_id IN (
    SELECT e.jugador_id
    FROM eventos_telemetria e
    JOIN tic t ON t.tic_id = e.tic_id
    GROUP BY e.jugador_id
    HAVING COUNT(t.tic_id) > (
        SELECT AVG(conteo)
        FROM (
            SELECT COUNT(t2.tic_id) AS conteo
            FROM eventos_telemetria e2
            JOIN tic t2 ON t2.tic_id = e2.tic_id
            GROUP BY e2.jugador_id
        ) AS promedios
    )
)
ORDER BY j.alias, i.numero_item;

```

26	Nicolas	Senti que progrese mientras jugaba Chocolate Doom.	7
27	Nicolas	Senti un sentido de logro mientras jugaba Chocolate Doom.	7
28	Nicolas	Con frecuencia senti que me faltaban las habilidades necesarias para Chocolate Doom.	6
29	Nicolas	Segui fallando en lograr lo que queria mientras jugaba Chocolate Doom.	3
30	Nicolas	Me senti decepcionado con mi desempeno en Chocolate Doom.	4
31	Nicolas	Senti que forme relaciones con otros jugadores y/o personajes en Chocolate Doom.	1
32	Nicolas	Al jugar Chocolate Doom, senti una conexion con otros, virtuales o reales.	1
33	Nicolas	Senti que otros jugadores y/o personajes en Chocolate Doom se preocupaban por mi.	1
34	Nicolas	Las interacciones con otros jugadores y/o personajes en Chocolate Doom me parecieron toxi...	1
35	Nicolas	La comunidad o el mundo virtual en Chocolate Doom me hizo sentir no bienvenido.	1
36	Nicolas	Los demas en Chocolate Doom fueron hostiles hacia mi.	1
Total rows: 36 Query complete 00:00:00.118			CR

-- Query 5: Sector mas visitado (hotspot) por episodio y mapa

```

SELECT
    ep.nombre_episodio,
    m.codigo_mapa,
    s.nombre_sector,
    COUNT(e.evento_id) AS visitas
FROM eventos_telemetria e
JOIN sectores s ON s.sector_id = e.sector_id
JOIN tic t ON t.tic_id = e.tic_id
JOIN sesiones_juego sj ON sj.session_id = t.session_id
JOIN mapas m ON m.mapa_id = sj.mapa_id
JOIN episodios ep ON ep.episodio_id = m.episodio_id

```

GROUP BY ep.nombre_episodio, m.codigo_mapa, s.nombre_sector
ORDER BY ep.nombre_episodio, m.codigo_mapa, visitas DESC;

Data Output Messages Notifications				
Showing rows: 1 to 812				
	nombre_episodio character varying (100)	codigo_mapa character varying (10)	nombre_sector character varying (50)	visitas bigint
1	THE CITY OF THE DAMN...	E1M1	(0,0)	620
2	THE CITY OF THE DAMN...	E1M1	(-7,-7)	324
3	THE CITY OF THE DAMN...	E1M1	(0,1)	292
4	THE CITY OF THE DAMN...	E1M1	(-7,9)	272
5	THE CITY OF THE DAMN...	E1M1	(-1,-6)	272
6	THE CITY OF THE DAMN...	E1M1	(6,0)	228
7	THE CITY OF THE DAMN...	E1M1	(0,-1)	228
8	THE CITY OF THE DAMN...	E1M1	(0,2)	224
9	THE CITY OF THE DAMN...	E1M1	(0,5)	216
10	THE CITY OF THE DAMN...	E1M1	(-4,-1)	216
11	THE CITY OF THE DAMN...	E1M1	(-3,3)	216
12	THE CITY OF THE DAMN...	E1M1	(1,0)	200
13	THE CITY OF THE DAMN...	E1M1	(-8,-9)	196
14	THE CITY OF THE DAMN...	E1M1	(-5,12)	184
15	THE CITY OF THE DAMN...	E1M1	(-3,-1)	184
16	THE CITY OF THE DAMN...	E1M1	(-7,-10)	176
17	THE CITY OF THE DAMN...	E1M1	(2,0)	168
18	THE CITY OF THE DAMN...	E1M1	(-2,7)	156
19	THE CITY OF THE DAMN...	E1M1	(-1,0)	152
20	THE CITY OF THE DAMN...	E1M1	(-1,-3)	148
21	THE CITY OF THE DAMN...	E1M1	(0,-5)	148
22	THE CITY OF THE DAMN...	E1M1	(-7,-8)	148
23	THE CITY OF THE DAMN...	E1M1	(3,7)	136
24	THE CITY OF THE DAMN...	E1M1	(0,3)	132
25	THE CITY OF THE DAMN...	E1M1	(-5,-9)	128
Total rows: 812 Query complete 00:00:00.227				

el sector más visitado en E1M1 (THE CITY OF THE DAMN...) es (0,0) con 620 visitas

-- Query 7: Puntaje UX promedio para jugadores con trayectoria mas corta por episodio
SELECT

```

ep.nombre_episodio,
j.alias,
ROUND(CAST(AVG(r.puntaje_total) AS NUMERIC), 2) AS puntaje_ux_promedio
FROM (
  SELECT
    e.jugadoresjugador_id,
    t.sesiones_juegosesion_id,
    SUM(
      SQRT(
        POWER(e.pos_x - prev.pos_x, 2) +
        POWER(e.pos_y - prev.pos_y, 2)
      )
    ) AS distancia_total
  FROM eventos_telemetria e
  JOIN tic t ON t.tic_id = e.tictic_id
  JOIN eventos_telemetria prev
    ON prev.jugadoresjugador_id = e.jugadoresjugador_id
    AND prev.tictic_id = e.tictic_id - 1
  GROUP BY e.jugadoresjugador_id, t.sesiones_juegosesion_id
) AS trayectoria

```



```

JOIN tic t ON t.sesiones_juegosesion_id = trayectoria.sesiones_juegosesion_id
JOIN sesiones_juego sj ON sj.sesion_id = t.sesiones_juegosesion_id
JOIN mapas m ON m.mapa_id = sj.mapasma_id
JOIN episodios ep ON ep.episodio_id = m.episodioepisodio_id
JOIN jugadores j ON j.jugador_id = trayectoria.jugadoresjugador_id
JOIN usuarios u ON u.user_id = j.usuariosuser_id
JOIN respuestas_ux r ON r.usuariosuser_id = u.user_id
WHERE trayectoria.distancia_total = (
    SELECT MIN(t2.distancia_total)
    FROM (
        SELECT
            e2.jugadoresjugador_id,
            t2.sesiones_juegosesion_id,
            SUM(
                SQRT(
                    POWER(e2.pos_x - prev2.pos_x, 2) +
                    POWER(e2.pos_y - prev2.pos_y, 2)
                )
            ) AS distancia_total
        FROM eventos_telemetria e2
        JOIN tic t2 ON t2.tic_id = e2.tictic_id
        JOIN eventos_telemetria prev2
            ON prev2.jugadoresjugador_id = e2.jugadoresjugador_id
            AND prev2.tictic_id = e2.tictic_id - 1
        JOIN sesiones_juego sj2 ON sj2.sesion_id = t2.sesiones_juegosesion_id
        JOIN mapas m2 ON m2.mapa_id = sj2.mapasma_id
        WHERE m2.episodioepisodio_id = ep.episodio_id
        GROUP BY e2.jugadoresjugador_id, t2.sesiones_juegosesion_id
    ) AS t2
)
GROUP BY ep.nombre_episodio, j.alias
ORDER BY ep.nombre_episodio;

```

	nombre_episodio character varying (100) 🔒	alias character varying (50) 🔒	puntaje_ux_promedio numeric 🔒
1	THE CITY OF THE DAMN...	Davidg	5.17
2	THE DOME OF D'SPARIL	Juanes	3.72
3	THE OSSUARY	Juan	3.22

```
-- Query 8: Distancia total y velocidad promedio por jugador
SELECT
  j.alias,
  ROUND(CAST(SUM(
    SQRT(
      POWER(e.pos_x - e2.pos_x, 2) +
      POWER(e.pos_y - e2.pos_y, 2)
    )
  ) AS NUMERIC), 2) AS distancia_total,
  ROUND(CAST(SUM(
    SQRT(
      POWER(e.pos_x - e2.pos_x, 2) +
      POWER(e.pos_y - e2.pos_y, 2)
    )
  ) / COUNT(e.evento_id) AS NUMERIC), 2) AS velocidad_promedio
FROM eventos_telemetria e
JOIN eventos_telemetria e2
  ON e2.jugadoresjugador_id = e.jugadoresjugador_id
  AND e2.tictic_id = e.tictic_id - 1
JOIN jugadores j ON j.jugador_id = e.jugadoresjugador_id
GROUP BY j.alias
ORDER BY distancia_total DESC;
```

	alias character varying (50) 🔒	distancia_total numeric 🔒	velocidad_promedio numeric 🔒
1	Nicolas	1691940.31	43.11
2	gaby.12	331390.15	41.94
3	Lindsay	119153.68	41.30
4	Juanes	108266.51	35.67
5	Juan	76697.19	29.22
6	Davidg	67549.87	45.46

5. Indices:

BD sin indice:

EXPLAIN ANALYZE

QUERY PLAN	text
2	Group Key: e.jugadoresjugador_id, t.lesiones_juegoseston_id
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=1300
5	-> Hash Join (cost=2141.18..3773.65 rows=13423 width=40) (actual time=15.578..77.971 rows=57176.00 loops=1)
6	Hash Cond: (e.tictic_id = t.tic_id)
7	Buffers: shared hit=1300
8	-> Hash Join (cost=1420.95..3018.18 rows=13423 width=40) (actual time=9.882..48.805 rows=57176.00 loops=1)
9	Hash Cond: ((e.jugadoresjugador_id = prev.jugadoresjugador_id) AND ((e.tictic_id = 1) = prev.tictic_id))
10	Buffers: shared hit=1160
11	-> Seq Scan on eventos_telemetria e (cost=0.00..916.38 rows=33638 width=24) (actual time=0.023..12.019 rows=33638.00 loops=1)
12	Buffers: shared hit=580
13	-> Hash (cost=916.38..916.38 rows=33638 width=24) (actual time=9.700..9.702 rows=33638.00 loops=1)
14	Buckets: 65536 Batches: 1 Memory Usage: 2352kB
15	Buffers: shared hit=580
16	-> Seq Scan on eventos_telemetria prev (cost=0.00..916.38 rows=33638 width=24) (actual time=0.007..5.048 rows=33638.00 loops=1)
17	Buffers: shared hit=580
18	-> Hash (cost=397.88..397.88 rows=25788 width=8) (actual time=5.542..5.543 rows=25788.00 loops=1)
19	Buckets: 32768 Batches: 1 Memory Usage: 1264kB
20	Buffers: shared hit=140
21	-> Seq Scan on tic t (cost=0.00..397.88 rows=25788 width=8) (actual time=0.027..2.494 rows=25788.00 loops=1)
22	Buffers: shared hit=140
23	Planning:
24	Buffers: shared hit=14 dirtied=1
25	Planning Time: 0.921 ms
26	Execution Time: 114.695 ms

SELECT j.alias, COUNT(*) AS
eventos
FROM eventos_telemetria e
JOIN jugadores j ON j.jugador_id =
e.jugadoresjugador_id
GROUP BY j.alias;
Sin índice: Execution Time: 12.713
ms

-- Índice 1
CREATE INDEX
idx_eventos_jugador ON

eventos_telemetria(jugadoresjugador_id);

-- EXPLAIN despues del indice
EXPLAIN ANALYZE
SELECT j.alias, COUNT(*) AS eventos
FROM eventos_telemetria e
JOIN jugadores j ON j.jugador_id = e.jugadoresjugador_id
GROUP BY j.alias;

Sin índice: 12.713 ms — Seq Scan
Con índice: 17.822 ms — Index Only Scan
Conclusión: Con solo 33,638 filas el índice no mejora porque la tabla es pequeña.
Con millones de filas de telemetría real el índice sería significativamente más
rápido.

-- Índice 2

QUERY PLAN	text
1	HashAggregate (cost=907.35..909.35 rows=200 width=126) (actual time=17.717..17.721 rows=6.00 loops=1)
2	Group Key: j.alias
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=2 read=30
5	-> Hash Join (cost=21.54..739.16 rows=33638 width=118) (actual time=0.796..11.111 rows=33638.00 loops=1)
6	Hash Cond: (e.jugadoresjugador_id = j.jugador_id)
7	Buffers: shared hit=2 read=30
8	-> Index Only Scan using idx_eventos_jugador on eventos_telemetria e (cost=0.29..628.86 rows=33638 width=4) (actual time=0.722..4.387 rows=33638.00 loops=1)
9	Heap Fetches: 0
10	Index Searches: 1
11	Buffers: shared hit=1 read=30
12	-> Hash (cost=15.00..15.00 rows=500 width=122) (actual time=0.058..0.059 rows=6.00 loops=1)
13	Buckets: 1024 Batches: 1 Memory Usage: 9kB
14	Buffers: shared hit=1
15	-> Seq Scan on jugadores j (cost=0.00..15.00 rows=500 width=122) (actual time=0.033..0.037 rows=6.00 loops=1)
16	Buffers: shared hit=1
17	Planning:
18	Buffers: shared hit=7 read=1
19	Planning Time: 0.401 ms
20	Execution Time: 17.822 ms

ANTES DE
CREARLO

CREATE INDEX idx_eventos_jugador_tic
ON eventos_telemetria(jugadoresjugador_id, tictic_id);

-- EXPLAIN antes del indice (correrlo ANTES de crear el indice)

EXPLAIN ANALYZE

```
SELECT
  e.jugadoresjugador_id,
  t.sesiones_juegosesion_id,
  SUM(
    SQRT(
      POWER(e.pos_x - prev.pos_x, 2) +
      POWER(e.pos_y - prev.pos_y, 2)
    )
  ) AS distancia_total
FROM eventos_telemetria e
JOIN tic t ON t.tic_id = e.tictic_id
JOIN eventos_telemetria prev
  ON prev.jugadoresjugador_id = e.jugadoresjugador_id
  AND prev.tictic_id = e.tictic_id - 1
GROUP BY e.jugadoresjugador_id, t.sesiones_juegosesion_id;
```

-- EXPLAIN despues del indice

EXPLAIN ANALYZE

```
SELECT
  e.jugadoresjugador_id,
  t.sesiones_juegosesion_id,
  SUM(
    SQRT(
      POWER(e.pos_x - prev.pos_x, 2) +
      POWER(e.pos_y - prev.pos_y, 2)
    )
  ) AS distancia_total
FROM eventos_telemetria e
JOIN tic t ON t.tic_id = e.tictic_id
JOIN eventos_telemetria prev
  ON prev.jugadoresjugador_id = e.jugadoresjugador_id
  AND prev.tictic_id = e.tictic_id - 1
GROUP BY e.jugadoresjugador_id, t.sesiones_juegosesion_id;
```

QUERY PLAN	
text	
2	Group Key: e.jugadoresjugador_id, t.sesiones_juegosesion_id
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=1300
5	-> Hash Join (cost=2141.18..3773.65 rows=13423 width=40) (actual time=12.923..77.466 rows=57176.00 loops=1)
6	Hash Cond: (e.tictic_id = t.tic_id)
7	Buffers: shared hit=1300
8	-> Hash Join (cost=1420.95..3018.18 rows=13423 width=40) (actual time=8.670..49.595 rows=57176.00 loops=1)
9	Hash Cond: ((e.jugadoresjugador_id = prev.jugadoresjugador_id) AND ((e.tictic_id - 1) = prev.tictic_id))
10	Buffers: shared hit=1160
11	-> Seq Scan on eventos_telemetria e (cost=0.00..916.38 rows=33638 width=24) (actual time=0.017..12.363 rows=33638.00 loops=1)
12	Buffers: shared hit=580
13	-> Hash (cost=916.38..916.38 rows=33638 width=24) (actual time=8.504..8.506 rows=33638.00 loops=1)
14	Buckets: 65536 Batches: 1 Memory Usage: 2352kB
15	Buffers: shared hit=580
16	-> Seq Scan on eventos_telemetria prev (cost=0.00..916.38 rows=33638 width=24) (actual time=0.005..4.228 rows=33638.00 loops=1)
17	Buffers: shared hit=580
18	-> Hash (cost=397.88..397.88 rows=25788 width=8) (actual time=4.163..4.164 rows=25788.00 loops=1)
19	Buckets: 32768 Batches: 1 Memory Usage: 1264kB
20	Buffers: shared hit=140
21	-> Seq Scan on tic t (cost=0.00..397.88 rows=25788 width=8) (actual time=0.017..1.865 rows=25788.00 loops=1)
22	Buffers: shared hit=140
23	Planning:
24	Buffers: shared hit=27 read=1
25	Planning Time: 0.922 ms
26	Execution Time: 115.500 ms

Indice 3

QUERY PLAN	
text	
38	-> Hash (cost=10.68..10.68 rows=48 width=227) (actual time=0.108..0.112 rows=48.00 loops=1)
39	Buckets: 1024 Batches: 1 Memory Usage: 11kB
40	Buffers: shared hit=13
41	-> Nested Loop (cost=0.16..10.68 rows=48 width=227) (actual time=0.047..0.097 rows=48.00 loops=1)
42	Buffers: shared hit=13
43	-> Seq Scan on mapas m (cost=0.00..1.48 rows=48 width=13) (actual time=0.019..0.024 rows=48.00 loops=1)
44	Buffers: shared hit=1
45	-> Memoize (cost=0.16..1.18 rows=1 width=222) (actual time=0.001..0.001 rows=1.00 loops=48)
46	Cache Key: m.episodiosepisodio_id
47	Cache Mode: logical
48	Hits: 42 Misses: 6 Evictions: 0 Overflows: 0 Memory Usage: 1kB
49	Buffers: shared hit=12
50	-> Index Scan using pk_episodios on episodios ep (cost=0.15..1.17 rows=1 width=222) (actual time=0.004..0.004 rows=1.00 loops=1)
51	Index Cond: (episodio_id = m.episodiosepisodio_id)
52	Index Searches: 6
53	Buffers: shared hit=12
54	-> Hash (cost=8.00..8.00 rows=400 width=11) (actual time=0.219..0.220 rows=417.00 loops=1)
55	Buckets: 1024 Batches: 1 Memory Usage: 27kB
56	Buffers: shared hit=4
57	-> Seq Scan on sectores s (cost=0.00..8.00 rows=400 width=11) (actual time=0.045..0.119 rows=417.00 loops=1)
58	Buffers: shared hit=4
59	Planning:
60	Buffers: shared hit=34 read=3
61	Planning Time: 2.889 ms
62	Execution Time: 256.389 ms

Antes de crearlo

```
CREATE INDEX idx_eventos_sector
ON eventos_telemetria(sectoressector_id, tictic_id);
```

-- EXPLAIN antes del indice

EXPLAIN ANALYZE

SELECT

```
    ep.nombre_episodio,
    m.codigo_mapa,
    s.nombre_sector,
    COUNT(e.evento_id) AS visitas
```

FROM eventos_telemetria e

JOIN sectores s ON s.sector_id = e.sectoressector_id

JOIN tic t ON t.tic_id = e.tictic_id

JOIN sesiones_juego sj ON sj.sesion_id = t.sesiones_juegosesion_id

JOIN mapas m ON m.mapa_id = sj.mapasmapa_id

JOIN episodios ep ON ep.episodio_id = m.episodiosepisodio_id

```
GROUP BY ep.nombre_episodio, m.codigo_mapa, s.nombre_sector
ORDER BY ep.nombre_episodio, m.codigo_mapa, visitas DESC;
```

-- EXPLAIN despues del indice

```
EXPLAIN ANALYZE
```

```
SELECT
```

```
    ep.nombre_episodio,
    m.codigo_mapa,
    s.nombre_sector,
    COUNT(e.evento_id) AS visitas
```

```
FROM eventos_telemetria e
```

```
JOIN sectores s ON s.sector_id = e.sectoressector_id
```

```
JOIN tic t ON t.tic_id = e.tictic_id
```

```
JOIN sesiones_juego sj ON sj.sesion_id = t.sesiones_juegosesion_id
```

```
JOIN mapas m ON m.mapa_id = sj.mapasmapa_id
```

```
JOIN episodios ep ON ep.episodio_id = m.episodioepisodio_id
```

```
GROUP BY ep.nombre_episodio, m.codigo_mapa, s.nombre_sector
```

```
ORDER BY ep.nombre_episodio, m.codigo_mapa, visitas DESC;
```

QUERY PLAN	text
38	-> Hash (cost=10.68..10.68 rows=48 width=227) (actual time=0.108..0.112 rows=48.00 loops=1)
39	Buckets: 1024 Batches: 1 Memory Usage: 11kB
40	Buffers: shared hit=13
41	-> Nested Loop (cost=0.16..10.68 rows=48 width=227) (actual time=0.047..0.097 rows=48.00 loops=1)
42	Buffers: shared hit=13
43	-> Seq Scan on mapas m (cost=0.00..1.48 rows=48 width=13) (actual time=0.019..0.024 rows=48.00 loops=1)
44	Buffers: shared hit=1
45	-> Memoize (cost=0.16..1.18 rows=1 width=222) (actual time=0.001..0.001 rows=1.00 loops=48)
46	Cache Key: m.episodioepisodio_id
47	Cache Mode: logical
48	Hits: 42 Misses: 6 Evictions: 0 Overflows: 0 Memory Usage: 1kB
49	Buffers: shared hit=12
50	-> Index Scan using pk_episodios on episodios ep (cost=0.15..1.17 rows=1 width=222) (actual time=0.004..0.004 rows=1.00 lo..)
51	Index Cond: (episodio_id = m.episodioepisodio_id)
52	Index Searches: 6
53	Buffers: shared hit=12
54	-> Hash (cost=8.00..8.00 rows=400 width=11) (actual time=0.219..0.220 rows=417.00 loops=1)
55	Buckets: 1024 Batches: 1 Memory Usage: 27kB
56	Buffers: shared hit=4
57	-> Seq Scan on sectores s (cost=0.00..8.00 rows=400 width=11) (actual time=0.045..0.119 rows=417.00 loops=1)
58	Buffers: shared hit=4
59	Planning:
60	Buffers: shared hit=34 read=3
61	Planning Time: 2.889 ms
62	Execution Time: 256.389 ms

Despues de crearlo

1. Trayectorias por jugador y sesión

```
CREATE VIEW vista_trayectorias AS
```

```
SELECT
```

```
    e.jugadoresjugador_id,
    t.sesiones_juegosesion_id,
    SUM(
        Sqrt(
            POWER(e.pos_x - prev.pos_x, 2) +
            POWER(e.pos_y - prev.pos_y, 2)
        )
    )
```

```

) AS distancia_total,
COUNT(e.evento_id) AS total_tics
FROM eventos_telemetria e
JOIN tic t ON t.tic_id = e.tictic_id
JOIN eventos_telemetria prev
ON prev.jugadoresjugador_id = e.jugadoresjugador_id
AND prev.tictic_id = e.tictic_id - 1
GROUP BY e.jugadoresjugador_id, t.sesiones_juegosesion_id;

```

	jugadoresjugador_id integer	sesiones_juegosesion_id integer	distancia_total double precision	total_tics bigint
1	4	5	108266.51396588508	3035
2	3	2	1524571.5124809113	35319
3	2	4	331390.14709281264	7902
4	5	6	76697.19381536319	2625
5	6	7	119153.6750628498	2885
6	1	3	67549.86898317991	1486
7	3	1	167368.79502620647	3924

2. Hotspots por episodio y mapa

```

CREATE VIEW vista_hotspots AS
SELECT
    ep.nombre_episodio,
    m.codigo_mapa,
    s.nombre_sector,
    COUNT(e.evento_id) AS visitas
FROM eventos_telemetria e
JOIN sectores s ON s.sector_id = e.sectoressector_id
JOIN tic t ON t.tic_id = e.tictic_id
JOIN sesiones_juego sj ON sj.sesion_id = t.sesiones_juegosesion_id
JOIN mapas m ON m.mapa_id = sj.mapasmapa_id
JOIN episodios ep ON ep.episodio_id = m.episodiosepisodio_id
GROUP BY ep.nombre_episodio, m.codigo_mapa, s.nombre_sector;

```

	nombre_episodio character varying (100)	codigo_mapa character varying (10)	nombre_sector character varying (50)	visitas bigint
1	THE CITY OF THE DAMN...	E1M1	(-1,1)	88
2	THE CITY OF THE DAMN...	E1M1	(-1,12)	4
3	THE CITY OF THE DAMN...	E1M1	(-1,2)	84
4	THE CITY OF THE DAMN...	E1M1	(-1,3)	148
5	THE CITY OF THE DAMN...	E1M1	(-1,5)	28
6	THE CITY OF THE DAMN...	E1M1	(-1,6)	272
7	THE CITY OF THE DAMN...	E1M1	(-1,7)	36
8	THE CITY OF THE DAMN...	E1M1	(-1,8)	152
9	THE CITY OF THE DAMN...	E1M1	(-1,1)	56
10	THE CITY OF THE DAMN...	E1M1	(-1,2)	72
11	THE CITY OF THE DAMN...	E1M1	(-1,3)	12
12	THE CITY OF THE DAMN...	E1M1	(-1,4)	24
13	THE CITY OF THE DAMN...	E1M1	(-1,5)	52
14	THE CITY OF THE DAMN...	E1M1	(-1,6)	64
15	THE CITY OF THE DAMN...	E1M1	(-1,7)	40
16	THE CITY OF THE DAMN...	E1M1	(-1,8)	80
17	THE CITY OF THE DAMN...	E1M1	(-10,5)	60
18	THE CITY OF THE DAMN...	E1M1	(-10,6)	48
19	THE CITY OF THE DAMN...	E1M1	(-10,7)	100
20	THE CITY OF THE DAMN...	E1M1	(-10,8)	36
21	THE CITY OF THE DAMN...	E1M1	(-11,5)	4
22	THE CITY OF THE DAMN...	E1M1	(-11,6)	28
23	THE CITY OF THE DAMN...	E1M1	(-11,7)	64
Total rows: 812 Query complete 00:00:00.278				

3. Duración promedio de sesiones por mapa

```
CREATE MATERIALIZED VIEW vistamat_duracion_sesiones AS
SELECT
    m.codigo_mapa,
    m.nombre_mapa,
    ROUND(AVG(EXTRACT(MINUTE FROM (sj.terminado_en -
sj.iniciado_en))), 2)
    AS duracion_promedio_minutos,
    COUNT(sj.sesion_id) AS total_sesiones
FROM sesiones_juego sj
JOIN mapas m ON m.mapa_id = sj.mapasmapa_id
GROUP BY m.codigo_mapa, m.nombre_mapa
ORDER BY duracion_promedio_minutos DESC;
```

	codigo_mapa character varying (10) 🔒	nombre_mapa character varying (100) 🔒	duracion_promedio_minutos numeric 🔒	total_sesiones bigint 🔒
1	E4M1	CATAFALQUE	30.00	1
2	E1M5	THE CITADEL	30.00	3
3	E3M1	THE STOREHOUSE	30.00	1
4	E1M1	THE DOCKS	30.00	2

Diccionario de Datos

El presente diccionario describe cada tabla de la base de datos del sistema de telemetría, detallando sus campos, tipos de dato, claves primarias y foráneas.

Tabla: usuario

Almacena los datos personales y de consentimiento de cada participante del estudio.

Campo	Tipo	PK	FK	Descripción
user_id	int4	✓		Identificador único del usuario (PK)
nombre_usuario	varchar(50)			Nombre del usuario en el sistema
email	varchar(100)			Correo electrónico del usuario
edad	int4			Edad del usuario en años
genero	varchar(20)			Género del usuario
nivel_experiencia	varchar(20)			Nivel de experiencia en videojuegos
creado_en	timestamp			Fecha y hora de registro

Campo	Tipo	PK	FK	Descripción
esta_activo	bool			Indica si el usuario está activo
firma_consentimiento_en	timestamp			Fecha de firma del consentimiento informado
consentimiento	varchar(1000)			Texto del consentimiento firmado

Tabla: jugador

Representa el perfil de juego de un usuario dentro del videojuego.

Campo	Tipo	PK	FK	Descripción
jugador_id	int4	✓		Identificador único del jugador (PK)
alias	varchar(50)			Nombre o apodo del jugador en el juego
creado_en	timestamp			Fecha de creación del perfil
usuariosuser_id	int4		✓	FK → usuario. Usuario real asociado al jugador

Tabla: instrumento_ux

Define los cuestionarios de evaluación de experiencia de usuario utilizados en el estudio.

Campo	Tipo	PK	FK	Descripción
instrumento_id	int4	✓		Identificador único del instrumento (PK)
nombre_instrumento	varchar(20)			Nombre del instrumento (ej. NASA-TLX, SUS)
version	varchar(10)			Versión del instrumento utilizado
descripcion	varchar(1000)			Descripción general del instrumento
escala_min	int4			Valor mínimo de la escala de respuesta
escala_max	int4			Valor máximo de la escala de respuesta
usuariosuser_id	int4		✓	FK → usuario. Usuario que administra el instrumento

Tabla: item_ux

Contiene cada pregunta individual perteneciente a un instrumento UX.

Campo	Tipo	PK	FK	Descripción
item_id	int4	✓		Identificador único del ítem (PK)
texto_item	varchar(1000)			Texto de la pregunta o afirmación
dimension	varchar(50)			Dimensión UX que evalúa el ítem
puntaje_invertido	bool			Indica si el puntaje se invierte al calcular
instrumentos_uxinstrumento_id	int4		✓	FK → instrumento_ux. Instrumento al que pertenece

Tabla: respuesta_ux

Registra las respuestas de cada usuario a los ítems de los cuestionarios UX.

Campo	Tipo	PK	FK	Descripción
respuesta_id	int4	✓		Identificador único de la respuesta (PK)
respondido_en	timestamp			Fecha y hora en que se respondió
puntaje	float4			Puntaje asignado por el usuario al ítem
item_uxitem_id	int4		✓	FK → item_ux. Ítem respondido
usuariosuser_id	int4		✓	FK → usuario. Usuario que respondió

Tabla: episodio

Representa una campaña o conjunto de niveles del videojuego.

Campo	Tipo	PK	FK	Descripción
episodio_id	int4	✓		Identificador único del episodio (PK)
numero_episodio	int4			Número secuencial del episodio
nombre_episodio	varchar(100)			Nombre descriptivo del episodio

Tabla: mapa

Describe cada escenario de juego con sus dimensiones y el episodio al que pertenece.

Campo	Tipo	PK	FK	Descripción
mapa_id	int4	✓		Identificador único del mapa (PK)
codigo_mapa	varchar(10)			Código corto de identificación del mapa
nombre_mapa	varchar(100)			Nombre del mapa en el juego
ancho_unidades	float4			Ancho del mapa en unidades del juego
alto_unidades	float4			Alto del mapa en unidades del juego
episodioepisodio_id	int4		✓	FK → episodio. Episodio al que pertenece el mapa

Tabla: sector

Subdivide un mapa en regiones geográficas definidas mediante coordenadas de bounding box.

Campo	Tipo	PK	FK	Descripción
sector_id	int4	✓		Identificador único del sector (PK)
nombre_sector	varchar(50)			Nombre descriptivo del sector
bbox_x_min	float4			Coordenada X mínima del bounding box
bbox_x_max	float4			Coordenada X máxima del bounding box
bbox_y_min	float4			Coordenada Y mínima del bounding box
bbox_y_max	float4			Coordenada Y máxima del bounding box
mapamapa_id	int4		✓	FK → mapa. Mapa al que pertenece el sector

Tabla: sesion_juego

Registra cada partida jugada, incluyendo el modo, dificultad y tiempos de inicio y fin.

Campo	Tipo	PK	FK	Descripción
sesion_id	int4	✓		Identificador único de la sesión (PK)
modo_juego	varchar(20)			Modo de juego seleccionado (ej. historia, desafío)
dificultad	varchar(20)			Nivel de dificultad de la sesión

Campo	Tipo	PK	FK	Descripción
iniciado_en	timestamp			Fecha y hora de inicio de la sesión
terminado_en	timestamp			Fecha y hora de fin de la sesión

Tabla: tic

Representa la unidad mínima de tiempo dentro de una sesión de juego.

Campo	Tipo	PK	FK	Descripción
tic_id	int4	✓		Identificador único del tic (PK)
numero_tic	int4			Número secuencial del tic en la sesión
timestamp_ms	int4			Marca de tiempo en milisegundos
sesiones_juegosesion_id	int4		✓	FK → sesion_juego. Sesión a la que pertenece el tic

Tabla: evento_telemetria

Captura el estado completo del jugador en cada instante de la partida: posición, movimiento y recursos.

Campo	Tipo	PK	FK	Descripción
evento_id	int4	✓		Identificador único del evento (PK)
momentum_dx	float4			Componente X del vector de momentum
pos_x	float4			Posición del jugador en el eje X
pos_y	float4			Posición del jugador en el eje Y
angulo_vision	float4			Ángulo de visión del jugador en grados
momentum_dy	float4			Componente Y del vector de momentum
salud	int4			Puntos de salud actuales del jugador
armadura	int4			Puntos de armadura actuales del jugador
municion	int4			Cantidad de munición disponible
jugadoresjugador_id	int4		✓	FK → jugador. Jugador que generó el evento

Campo	Tipo	PK	FK	Descripción
sectoressector_id	int4		✓	FK → sector. Sector donde ocurrió el evento
tictic_id	int4		✓	FK → tic. Unidad de tiempo del evento

Mecanismo de Recreación del Schema

— setup_database.bat

Para facilitar la recreación del entorno de base de datos, se desarrolló un script de Windows (setup_database.bat) que automatiza la configuración completa del proyecto en cinco pasos: Paso 1 — Creación de tablas: ejecuta el DDL (DDLdoomchocolate.sql) que define todas las tablas, constraints y llaves del schema. Si las tablas ya existen, las omite sin interrumpir el proceso.

Paso 2 — Carga de datos base: ejecuta usuarios_y_respuestas.sql, que inserta los episodios, mapas, usuarios, jugadores, sesiones y respuestas de la encuesta BANGS.

Paso 3 — Creación del staging: crea la tabla staging_telemetria usada como zona temporal durante el ETL. Si ya existe, la omite.

Paso 4 — Carga de telemetría: indica al usuario qué archivos de telemetría debe cargar manualmente usando CargueTXTaStagingTelemetry.sql e InsertarDatosTelemetry.sql, especificando el jugador, mapa y ruta de cada archivo.

Paso 5 — Verificación: muestra un conteo de filas por tabla para confirmar que los datos fueron cargados correctamente.

Requisitos: PostgreSQL instalado y en el PATH del sistema, y todos los archivos .sql en la misma carpeta que el .bat.

```
[1/5] Creando tablas...
psql:DDLdoomchocolate.sql:11: ERROR:  la relación «episodios» ya existe
psql:DDLdoomchocolate.sql:25: ERROR:  la relación «mapas» ya existe
psql:DDLdoomchocolate.sql:42: ERROR:  la relación «sectores» ya existe
psql:DDLdoomchocolate.sql:57: ERROR:  la relación «sesiones_juego» ya existe
psql:DDLdoomchocolate.sql:71: ERROR:  la relación «tic» ya existe
psql:DDLdoomchocolate.sql:89: ERROR:  la relación «usuarios» ya existe
psql:DDLdoomchocolate.sql:103: ERROR: la relación «jugadores» ya existe
psql:DDLdoomchocolate.sql:121: ERROR: la relación «instrumentos_ux» ya existe
psql:DDLdoomchocolate.sql:137: ERROR: la relación «items_ux» ya existe
psql:DDLdoomchocolate.sql:157: ERROR: la relación «respuestas_ux» ya existe
psql:DDLdoomchocolate.sql:184: ERROR: la relación «eventos_telemetria» ya existe
psql:DDLdoomchocolate.sql:210: ERROR: la relación «staging_telemetria» ya existe
psql:DDLdoomchocolate.sql:224: ERROR: la relación «registros_auditoria» ya existe
psql:DDLdoomchocolate.sql:234: ERROR: llave duplicada viola restricción de unicidad «pk_instrumentos_ux»
DETALLE: Ya existe la llave (instrumento_id)=(1).
psql:DDLdoomchocolate.sql:257: ERROR: llave duplicada viola restricción de unicidad «pk_items_ux»
DETALLE: Ya existe la llave (item_id)=(1).
Tablas creadas correctamente.

[2/5] Cargando usuarios, jugadores, episodios y mapas...
psql:usuarios_y_respuestas.sql:15: ERROR: llave duplicada viola restricción de unicidad «pk_usuarios»
DETALLE: Ya existe la llave (user_id)=(1).
psql:usuarios_y_respuestas.sql:24: ERROR: llave duplicada viola restricción de unicidad «pk_jugadores»
DETALLE: Ya existe la llave (jugador_id)=(1).
psql:usuarios_y_respuestas.sql:35: ERROR: llave duplicada viola restricción de unicidad «pk_episodios»
DETALLE: Ya existe la llave (episodio_id)=(1).
psql:usuarios_y_respuestas.sql:66: ERROR: llave duplicada viola restricción de unicidad «pk_mapas»
DETALLE: Ya existe la llave (mapa_id)=(1).
```

```
[4/5] Cargando telemetria...
Nota: Edita las variables en CargueTXTaStagingTelemetry.sql
para cada archivo y luego corre InsertarDatosTelemetry.sql

Archivos a cargar:
- nicolas_clean.txt      (E1M1, Episodio 1)
- Davidg_clean.txt      (E1M5, Episodio 1)
- gaby_12_clean.txt      (E1M5, Episodio 1)
- Juanes_clean.txt       (E3M1, Episodio 3)
- Juan_clean.txt         (E4M1, Episodio 4)
- Lindsay_clean.txt      (E1M5, Episodio 1)

[5/5] Verificando datos cargados...
      tabla      | filas
-----|-----
episodios         |      6
mapas             |     48
usuarios          |      6
jugadores         |      6
sesiones_juego    |      7
sectores          |    417
tic               |   25788
eventos_telemetria |   33638
respuestas_ux     |     108
```

```

psql:usuarios_y_respuestas.sql:149: ERROR: no existe la columna «instrumentos_uxinstrumento_id» en la relación «respues
tas_ux»
LÍNEA 1: ...total, usuariosuser_id, sesiones_juegosesion_id, instrument...

psql:usuarios_y_respuestas.sql:161: ERROR: no existe la columna «instrumentos_uxinstrumento_id» en la relación «respues
tas_ux»
LÍNEA 1: ...total, usuariosuser_id, sesiones_juegosesion_id, instrument...

psql:usuarios_y_respuestas.sql:182: ERROR: no existe la columna «instrumentos_uxinstrumento_id» en la relación «respues
tas_ux»
LÍNEA 1: ...total, usuariosuser_id, sesiones_juegosesion_id, instrument...

psql:usuarios_y_respuestas.sql:203: ERROR: no existe la columna «instrumentos_uxinstrumento_id» en la relación «respues
tas_ux»
LÍNEA 1: ...total, usuariosuser_id, sesiones_juegosesion_id, instrument...

psql:usuarios_y_respuestas.sql:224: ERROR: no existe la columna «instrumentos_uxinstrumento_id» en la relación «respues
tas_ux»
LÍNEA 1: ...total, usuariosuser_id, sesiones_juegosesion_id, instrument...

Usuarios y respuestas cargados correctamente.

[3/3] Creando tabla staging...
NOTICE: la relación «staging-telemetria» ya existe, omitiendo
CREATE TABLE
Staging creado correctamente.

```

En conclusión, la base de datos diseñada constituye una solución robusta y normalizada para el almacenamiento y consulta de datos de telemetría, con capacidad de escalar a múltiples sesiones, jugadores e instrumentos de evaluación, y con una estructura que facilita el análisis tanto cuantitativo como cualitativo del comportamiento del usuario en entornos de videojuego.